

1 - Criação e Configuração do Banco de Dados

Deve se dar privilégios ao banco de dados que irá usar

```
aspnet-Project00_SqlServer - Data - CREATE DATABASE mydatabase.sql
$(plug) Conectar ao MSSQL
1 CREATE DATABASE mydatabase;
2
3 USE mydatabase;
4
5 -- Autorizar o usuário do Windows a ser o proprietário do banco de dados (Permissão)
6 ALTER AUTHORIZATION ON DATABASE::mydatabase TO [NOTE-MAIQUELJ\Maiquel];
7
8 -- Verificar se o usuário tem permissão de db_owner
9 SELECT name AS mydatabase, SUSER_SNAME(owner_sid) AS Dono_Atual_Proprietario FROM sys.databases WHERE name = 'mydatabase';
10
11 -- Vasculha a tabela Usuarios
12 SELECT * FROM Usuarios;
```

Segurança Integrada: Ao usar o `ALTER AUTHORIZATION` para o seu usuário do Windows, eliminamos a necessidade de salvar senhas em texto puro no código, o que é uma boa prática de segurança.

Privilégio `db_owner`: Esse comando concede automaticamente o papel de `db_owner`. Isso significa que o seu código ASP.NET tem permissão total para realizar operações de **CRUD** (Create, Read, Update, Delete).

2 - appsettings.json

É aqui que definimos onde o banco está e quem tem permissão para entrar.

```
{
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=.\SQLEXPRESS;Database=mydatabase;Integrated Security=True;TrustServerCertificate=True;"
  }
}
```

3 - Program.cs

Deve - se fazer a configuração correta no **Program.cs** para que tudo funcione

```
using Microsoft.EntityFrameworkCore; // Importar o Entity Framework Core para configurar o contexto do banco de dados
using Asp.Net_Projeto08_SQLServer.Data; // Importar o contexto do banco de dados para registrar o serviço no contêiner de injeção de dependência

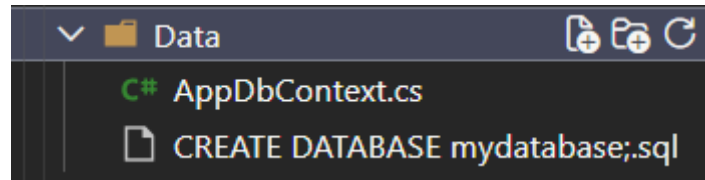
var builder = WebApplication.CreateBuilder(args);

// Pega a string de conexão do arquivo appsettings.json
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");

// Usa a permissão do SQL Server para acessar o banco de dados (Cujo criei no proprio Banco de Dados do SQL Server)
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));
```

4 - AppDbContext.cs

Criamos uma pasta chamado **Data** e criamos um arquivos chamado **AppDbContext.cs** dentro nela criamos a “**ponte**” de comunicação entre o **C#** e o **Banco De Dados**



```
1 using Microsoft.EntityFrameworkCore;
2 using Asp.Net_Projeto08_SQLServer.Models;
3
4 namespace Asp.Net_Projeto08_SQLServer.Data;
5
6 7 referências
7 public class AppDbContext : DbContext
8 {
9     // Criar uma tabela Usuario no banco de dados
10     2 referências
11     public DbSet<Usuario> Usuarios { get; set; }
12
13     // Configurar a string de conexão com o banco de dados SQL Server
14     0 referências
15     public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
16     {
17     }
18 }
```

`public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options)` → Esse é o código principal responsável pela ponte.

5 - Lógica do HomeController.cs

```
1 referência
public class HomeController : Controller
{
    // Variavel Global para acessar o banco de dados
    4 referências
    private readonly ApplicationDbContext _context;

    // Injetar o contexto do banco de dados no controlador
    0 referências
    public HomeController(ApplicationDbContext context)
    {
        _context = context;
    }
}
```

`private readonly ApplicationDbContext _context;` → Variável global responsável por acessar o Banco de Dados.

`public HomeController(ApplicationDbContext context)` → Função responsável por enviar os dados para o arquivo `ApplicationDbContext.cs`; ou seja, é por onde os dados são enviados para o Banco de Dados.

O Resto segue Normalmente com Models , Views e etc.